

Workshop 2: Adding Sorting and Pagination to CRUD Application

Prerequisites

Before starting this workshop, you must:

1. Complete **Workshop 1** to set up the base CRUD project
2. Have a working application with Create, Read, Update, Delete functionality
3. Understand the API-first design approach from Workshop 1

Part 0: Understanding the New Features - API Design First

Learning Objective

Before writing any code, you must:

1. Identify what new APIs are needed
2. Design the API endpoints and parameters
3. Plan how HTML pages will call these APIs
4. Understand the data flow

Golden Rule: Never write code before designing the API.

Table 1: New API Requirements Analysis

Feature	What User Sees	What API Needs	Why
Sort by Name	Click "Name" button → items sorted A-Z	API must accept <code>sort=name</code> parameter	Server needs to know which field to sort by
Sort by Price	Click "Price" button → items sorted low-high	API must accept <code>sort=price</code> parameter	Server needs to know which field to sort by
Default Order	Click "Default" button → items in original order	API must accept <code>sort=default</code> or no parameter	Server needs to know to use original order
Page 1	Click "1" → shows items 1-5	API must accept <code>page=1</code> parameter	Server needs to know which page to return
Page 2	Click "2" → shows items 6-10	API must accept <code>page=2</code> parameter	Server needs to know which page to return
Next Page	Click "Next" → shows next 5 items	API must calculate <code>page=current+1</code>	Server needs to know to increment page
Previous Page	Click "Previous" → shows previous 5 items	API must calculate <code>page=current-1</code>	Server needs to know to decrement page

Key Insight: Sorting and pagination are **query parameters** added to the existing **GET /items/** endpoint, not new endpoints.

Table 2: Updated API Endpoint Design

Method	Path	Query Parameters	Purpose	Response	Notes
GET	/items/	sort (optional) page (optional)	List items with sorting and pagination	HTML page with filtered items	Modified from Workshop 1
GET	/items/new	None	Show create form	HTML form	Unchanged
POST	/items/	None (form data in body)	Create new item	Redirect to /items/	Unchanged
GET	/items/edit/{item_id}	None	Show edit form	HTML form	Unchanged
POST	/items/{item_id}	None (form data in body)	Update item	Redirect to /items/	Unchanged
POST	/items/delete/{item_id}	None	Delete item	Redirect to /items/	Unchanged

What Changed?

- Only **GET /items/** is modified
- All other endpoints remain the same
- New parameters: **sort** and **page**

Table 3: Query Parameter Design

Parameter	Type	Possible Values	Default	Example URL	What It Does
sort	string	name, price, default	default	/items/?sort=name	Determines sort order
page	integer	1, 2, 3, ...	1	/items/?page=2	Determines which page to show
Combined	-	-	-	/items/?sort=price&page=2	Sort by price AND show page 2

Important URL Rules:

- First parameter uses **?** → **/items/?sort=name**

- Additional parameters use & → `/items/?sort=name&page=2`
- Parameters can be in any order → `/items/?page=2&sort=name` (same result)

Table 4: How Links Call the API

User Clicks	HTML Link	API Called	Server Action
"Name" button	<code></code>	<code>GET /items/?sort=name&page=1</code>	Sort by name, show page 1
"Price" button	<code></code>	<code>GET /items/?sort=price&page=1</code>	Sort by price, show page 1
"Default" button	<code></code>	<code>GET /items/?sort=default&page=1</code>	Original order, show page 1
"Next" button	<code></code>	<code>GET /items/?sort=name&page=3</code>	Keep sort, next page
"Previous" button	<code></code>	<code>GET /items/?sort=name&page=1</code>	Keep sort, previous page
Page number "2"	<code></code>	<code>GET /items/?sort=price&page=2</code>	Keep sort, jump to page 2

Key Insight: Links are just URLs that trigger GET requests. The `href` attribute is the API call.

Table 5: Data Flow for Sorting

```

User clicks "Sort by Price"
↓
Browser sends: GET /items/?sort=price&page=1
↓
FastAPI receives: sort="price", page=1
↓
items.py loads all items from JSON
↓
items.py sorts items by price (ascending)
↓
items.py calculates: show items 0-4 (page 1, 5 per page)
↓
items.py passes to template:
- items (sorted and sliced)
- current_sort="price"
- current_page=1

```

```

- total_pages=3
↓
Template renders HTML with:
- Sorted items displayed
- "Price" button highlighted
- Pagination links with sort=price preserved
↓
Browser displays the page

```

Table 6: Data Flow for Pagination

```

User clicks "Next" (currently on page 1, sorted by name)
↓
Browser sends: GET /items/?sort=name&page=2
↓
FastAPI receives: sort="name", page=2
↓
items.py loads all items from JSON
↓
items.py sorts items by name (because sort="name")
↓
items.py calculates: show items 5-9 (page 2, 5 per page)
↓
items.py passes to template:
- items (sorted and sliced for page 2)
- current_sort="name"
- current_page=2
- total_pages=3
↓
Template renders HTML with:
- Items 6-10 displayed
- "Name" button still highlighted
- Page 2 highlighted in pagination
- Previous/Next buttons updated
↓
Browser displays the page

```

Table 7: Template Variable Requirements

The `items_list.html` template now needs these variables:

Variable	Type	Example Value	Purpose	Where It Comes From
<code>request</code>	Request	(FastAPI object)	Required by Jinja2	FastAPI automatically
<code>items</code>	list	<code>[{id:0, name:"Laptop",...},</code>	Items to display (already sorted	Calculated in <code>items.py</code>

Variable	Type	Example Value	Purpose	Where It Comes From
		...]	and paginated)	
current_sort	string	"name" or "price" or "default"	Which sort is active	From query parameter
current_page	integer	2	Which page is active	From query parameter
total_pages	integer	3	How many pages total	Calculated: <code>ceil(total_items / items_per_page)</code>
items_per_page	integer	5	How many items per page	Hardcoded constant

How Template Uses These:

- `items` → Loop to display items
- `current_sort` → Highlight active sort button
- `current_page` → Highlight active page number
- `total_pages` → Generate page number links
- `current_sort` + `current_page` → Build pagination links that preserve sort

Table 8: Pagination Math

Given:

- Total items: 12
- Items per page: 5

Page	Calculation	Items Shown	Array Slice
1	$start = (1-1) * 5 = 0$ $end = 1 * 5 = 5$	Items 1-5	<code>items[0:5]</code>
2	$start = (2-1) * 5 = 5$ $end = 2 * 5 = 10$	Items 6-10	<code>items[5:10]</code>
3	$start = (3-1) * 5 = 10$ $end = 3 * 5 = 15$	Items 11-12	<code>items[10:15]</code>

Formula:

```
start = (page - 1) * items_per_page
end = page * items_per_page
items_to_show = all_items[start:end]
```

Total Pages Calculation:

```
import math
total_pages = math.ceil(total_items / items_per_page)
# Example: ceil(12 / 5) = ceil(2.4) = 3 pages
```

Table 9: Common Bugs and How to Avoid Them

Bug	Symptom	Cause	Solution
Sort resets when changing page	Click "Next" → sort changes to default	Pagination links don't include <code>sort</code> parameter	Always include <code>sort={{ current_sort }}</code> in pagination links
Page resets when changing sort	Click "Sort by Price" → goes back to page 1	This is actually correct behavior!	When changing sort, always reset to <code>page=1</code>
"Previous" button on page 1	Clicking "Previous" on page 1 → error or page 0	No validation for minimum page	Disable/hide "Previous" when <code>current_page == 1</code>
"Next" button on last page	Clicking "Next" on page 3 → error or empty page	No validation for maximum page	Disable/hide "Next" when <code>current_page == total_pages</code>
Empty page	Page shows no items	Requesting page beyond total pages	Redirect to page 1 if <code>page > total_pages</code>
Negative page	URL has <code>page=-1</code> or <code>page=0</code>	User manually edited URL	Validate: if <code>page < 1</code> , set <code>page = 1</code>
Invalid sort value	URL has <code>sort=invalid</code>	User manually edited URL	Validate: if sort not in allowed values, use <code>default</code>

Quiz: Test Your Understanding

Before proceeding to implementation, answer these:

Question 1: If a user is on page 2 with items sorted by price, and clicks "Next", what should the URL be?

► Click to reveal answer

Question 2: What variables must the FastAPI route pass to the template for pagination to work?

► Click to reveal answer

Question 3: If you have 23 items and show 5 per page, how many pages are there?

► Click to reveal answer

Question 4: Why do we reset to page 1 when changing sort order?

► Click to reveal answer

Question 5: What's the difference between these two URLs?

- `/items/?sort=name&page=2`
- `/items/?page=2&sort=name`

► Click to reveal answer

Part 1: Update the Backend API

Step 1.1: Modify `app/items.py` - Add Sorting and Pagination Logic

Replace the existing `list_items` function with this enhanced version:

```
import math
from fastapi import APIRouter, Request, Form, Query
from fastapi.templating import Jinja2Templates
from fastapi.responses import RedirectResponse
import json
from pathlib import Path

router = APIRouter(prefix="/items", tags=["items"])
templates = Jinja2Templates(directory="app/templates")

DATA_FILE = Path("app/data/items.json")
ITEMS_PER_PAGE = 5 # Constant: how many items per page

def load_items():
    """Load items from JSON file"""
    with open(DATA_FILE, "r") as f:
        return json.load(f)

def save_items(items):
    """Save items to JSON file"""
    with open(DATA_FILE, "w") as f:
        json.dump(items, f, indent=2)

@router.get("/")
async def list_items(
    request: Request,
    sort: str = Query("default", regex="^(default|name|price)$"),
    page: int = Query(1, ge=1)
):
    """
    Display items with sorting and pagination

    Query Parameters:
    - sort: "default" | "name" | "price" (default: "default")
    - page: integer >= 1 (default: 1)
    """
    # Step 1: Load all items
    items = load_items()
```

```

# Step 2: Apply sorting
if sort == "name":
    items = sorted(items, key=lambda x: x["name"].lower())
elif sort == "price":
    items = sorted(items, key=lambda x: x["price"])
# else: sort == "default", keep original order

# Step 3: Calculate pagination
total_items = len(items)
total_pages = math.ceil(total_items / ITEMS_PER_PAGE)

# Step 4: Validate page number
if page > total_pages and total_pages > 0:
    # Redirect to last valid page
    return HttpResponseRedirect(
        url=f"/items/?sort={sort}&page={total_pages}",
        status_code=303
    )

# Step 5: Slice items for current page
start = (page - 1) * ITEMS_PER_PAGE
end = start + ITEMS_PER_PAGE
paginated_items = items[start:end]

# Step 6: Pass everything to template
return templates.TemplateResponse("items_list.html", {
    "request": request,
    "items": paginated_items,
    "current_sort": sort,
    "current_page": page,
    "total_pages": total_pages,
    "items_per_page": ITEMS_PER_PAGE
})

```

Code Explanation:

Line-by-line breakdown:

```
sort: str = Query("default", regex="^(default|name|price)$"),
```

- `Query("default", ...)` → Default value is "default" if not provided
- `regex="^(default|name|price)$"` → Only allow these three values
- If user sends `?sort=invalid`, FastAPI returns 422 error automatically

```
page: int = Query(1, ge=1)
```

- `Query(1, ...)` → Default value is 1 if not provided
- `ge=1` → "Greater than or equal to 1" (no negative or zero pages)

- If user sends `?page=0`, FastAPI returns 422 error automatically

```
if sort == "name":
    items = sorted(items, key=lambda x: x["name"].lower())
```

- `sorted()` → Returns new sorted list (doesn't modify original)
- `key=lambda x: x["name"].lower()` → Sort by name, case-insensitive
- `.lower()` → "Zebra" comes after "apple" (not before)

```
elif sort == "price":
    items = sorted(items, key=lambda x: x["price"])
```

- Sort by price numerically (10.00 < 100.00)

```
total_pages = math.ceil(total_items / ITEMS_PER_PAGE)
```

- `math.ceil()` → Round up (2.1 becomes 3)
- Example: 12 items ÷ 5 per page = 2.4 → 3 pages

```
if page > total_pages and total_pages > 0:
    return RedirectResponse(...)
```

- If user requests page 10 but only 3 pages exist → redirect to page 3
- `total_pages > 0` → Avoid redirect when there are no items

```
start = (page - 1) * ITEMS_PER_PAGE
end = start + ITEMS_PER_PAGE
paginated_items = items[start:end]
```

- Page 1: `start=0, end=5` → `items[0:5]`
- Page 2: `start=5, end=10` → `items[5:10]`
- Python slicing handles out-of-range gracefully (no error if `end > length`)

Step 1.2: Keep All Other Routes Unchanged

Important: The create, edit, update, and delete routes from Workshop 1 remain exactly the same. Only the `list_items` function changed.

Your complete `app/items.py` should now have:

-  `list_items()` - **MODIFIED** (sorting + pagination)

- ☒ `new_item_form()` - unchanged
- ☒ `create_item()` - unchanged
- ☒ `edit_item_form()` - unchanged
- ☒ `update_item()` - unchanged
- ☒ `delete_item()` - unchanged

Part 2: Update the Frontend Template

Step 2.1: Create Sorting and Pagination UI

Replace `app/templates/items_list.html` with this enhanced version:

```
{% extends "base.html" %}

{% block title %}Items List{% endblock %}

{% block content %}
<div class="container">
  <h1>Items List</h1>

  <!-- Sorting Controls -->
  <div class="sort-controls">
    <span>Sort by:</span>
    <a href="/items/?sort=default&page=1"
      class="btn btn-sm {% if current_sort == 'default' %}btn-active{% endif
%}">
      Default
    </a>
    <a href="/items/?sort=name&page=1"
      class="btn btn-sm {% if current_sort == 'name' %}btn-active{% endif
%}">
      Name
    </a>
    <a href="/items/?sort=price&page=1"
      class="btn btn-sm {% if current_sort == 'price' %}btn-active{% endif
%}">
      Price
    </a>
  </div>

  <!-- Items Table -->
  <table>
    <thead>
      <tr>
        <th>ID</th>
        <th>Name</th>
        <th>Price</th>
        <th>Special Offer</th>
        <th>Actions</th>
      </tr>
    </thead>
```

```

<tbody>
  {% if items %}
    {% for item in items %}
      <tr>
        <td>{{ item.id }}</td>
        <td>{{ item.name }}</td>
        <td>${{ "%.2f"|format(item.price) }}</td>
        <td>
          {% if item.is_offer %}
            <span class="badge badge-success">Yes</span>
          {% else %}
            <span class="badge">No</span>
          {% endif %}
        </td>
        <td>
          <a href="/items/edit/{{ item.id }}" class="btn btn-sm">Edit</a>
          <form action="/items/delete/{{ item.id }}" method="post"
            style="display:inline;">
            <button type="submit" class="btn btn-sm btn-danger"
              onclick="return confirm('Delete this item?')">
              Delete
            </button>
          </form>
        </td>
      </tr>
    {% endfor %}
  {% else %}
    <tr>
      <td colspan="5" style="text-align: center;">No items
        found</td>
    </tr>
  {% endif %}
</tbody>
</table>

<!-- Pagination Controls -->
{% if total_pages > 1 %}
<div class="pagination">
  <!-- Previous Button -->
  {% if current_page > 1 %}
    <a href="/items/?sort={{ current_sort }}&page={{ current_page - 1 }}"
      class="btn btn-sm">
      < Previous
    </a>
  {% else %}
    <span class="btn btn-sm btn-disabled">< Previous</span>
  {% endif %}

  <!-- Page Numbers -->
  {% for page_num in range(1, total_pages + 1) %}
    {% if page_num == current_page %}
      <span class="btn btn-sm btn-active">{{ page_num }}</span>
    {% else %}

```

```

        <a href="/items/?sort={{ current_sort }}&page={{ page_num }}"
          class="btn btn-sm"
          {{ page_num }}
        </a>
      {% endif %}
    {% endfor %}

    <!-- Next Button -->
    {% if current_page < total_pages %}
      <a href="/items/?sort={{ current_sort }}&page={{ current_page + 1 }}"
        class="btn btn-sm"
        Next →
      </a>
    {% else %}
      <span class="btn btn-sm btn-disabled">Next →</span>
    {% endif %}
  </div>

  <!-- Pagination Info -->
  <p style="text-align: center; color: #666; margin-top: 10px;">
    Page {{ current_page }} of {{ total_pages }}
    ({{ items|length }} items on this page)
  </p>
  {% endif %}

  <!-- Action Buttons -->
  <div style="margin-top: 20px;">
    <a href="/items/new" class="btn">Add New Item</a>
    <a href="/" class="btn">Back to Home</a>
  </div>
</div>
{% endblock %}

```

Template Explanation:

Sorting Controls:

```

<a href="/items/?sort=name&page=1"
  class="btn btn-sm {% if current_sort == 'name' %}btn-active{% endif %}">
  Name
</a>

```

- `href="/items/?sort=name&page=1"` → API call with query parameters
- `{% if current_sort == 'name' %}btn-active{% endif %}` → Highlight active sort
- Always reset to `page=1` when changing sort (intentional design choice)

Pagination - Previous Button:

```
{% if current_page > 1 %}
    <a href="/items/?sort={{ current_sort }}&page={{ current_page - 1 }}"
      class="btn btn-sm">
        ← Previous
    </a>
{% else %}
    <span class="btn btn-sm btn-disabled">← Previous</span>
{% endif %}
```

- `{% if current_page > 1 %}` → Only show clickable button if not on first page
- `sort={{ current_sort }}` → **Critical:** Preserve current sort
- `page={{ current_page - 1 }}` → Go to previous page
- `{% else %}` → Show disabled button on page 1

Pagination - Page Numbers:

```
{% for page_num in range(1, total_pages + 1) %}
    {% if page_num == current_page %}
        <span class="btn btn-sm btn-active">{{ page_num }}</span>
    {% else %}
        <a href="/items/?sort={{ current_sort }}&page={{ page_num }}"
          class="btn btn-sm">
            {{ page_num }}
        </a>
    {% endif %}
{% endfor %}
```

- `range(1, total_pages + 1)` → Generate numbers 1, 2, 3, ..., total_pages
- `{% if page_num == current_page %}` → Highlight current page (not clickable)
- `{% else %}` → Other pages are clickable links
- Again, preserve `sort={{ current_sort }}`

Pagination - Next Button:

```
{% if current_page < total_pages %}
    <a href="/items/?sort={{ current_sort }}&page={{ current_page + 1 }}"
      class="btn btn-sm">
        Next →
    </a>
{% else %}
    <span class="btn btn-sm btn-disabled">Next →</span>
{% endif %}
```

- `{% if current_page < total_pages %}` → Only show clickable button if not on last page
- `page={{ current_page + 1 }}` → Go to next page
- `{% else %}` → Show disabled button on last page

Step 2.2: Update CSS for New UI Elements

Add to `app/static/css/style.css`:

```
/* Sorting Controls */
.sort-controls {
  margin: 20px 0;
  padding: 15px;
  background: #f5f5f5;
  border-radius: 5px;
  display: flex;
  align-items: center;
  gap: 10px;
}

.sort-controls span {
  font-weight: bold;
  color: #333;
}

/* Pagination */
.pagination {
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 5px;
  margin: 20px 0;
  flex-wrap: wrap;
}

/* Button Variants */
.btn-sm {
  padding: 5px 12px;
  font-size: 14px;
}

.btn-active {
  background: #007bff;
  color: white;
  cursor: default;
}

.btn-active:hover {
  background: #007bff;
  transform: none;
}

.btn-disabled {
  background: #e0e0e0;
  color: #999;
  cursor: not-allowed;
}
```

```
}

.btn-disabled:hover {
  background: #e0e0e0;
  transform: none;
}

/* Badge Styles */
.badge {
  display: inline-block;
  padding: 3px 8px;
  border-radius: 3px;
  font-size: 12px;
  background: #6c757d;
  color: white;
}

.badge-success {
  background: #28a745;
}

/* Responsive Design */
@media (max-width: 768px) {
  .sort-controls {
    flex-direction: column;
    align-items: flex-start;
  }

  .pagination {
    font-size: 12px;
  }

  .btn-sm {
    padding: 4px 8px;
    font-size: 12px;
  }
}
```

CSS Explanation:

- `.sort-controls` → Horizontal bar with sort buttons
- `.pagination` → Horizontal bar with page buttons
- `.btn-active` → Highlighted button (current sort/page)
- `.btn-disabled` → Grayed-out button (can't click)
- `@media (max-width: 768px)` → Mobile-friendly layout

Part 3: Add Test Data

Step 3.1: Create More Items for Testing

To properly test pagination, you need more than 5 items.

Replace `app/data/items.json` with this:

```
[
  {
    "id": 0,
    "name": "Laptop",
    "price": 999.99,
    "is_offer": true
  },
  {
    "id": 1,
    "name": "Mouse",
    "price": 25.50,
    "is_offer": false
  },
  {
    "id": 2,
    "name": "Keyboard",
    "price": 75.00,
    "is_offer": true
  },
  {
    "id": 3,
    "name": "Monitor",
    "price": 299.99,
    "is_offer": false
  },
  {
    "id": 4,
    "name": "Webcam",
    "price": 89.99,
    "is_offer": true
  },
  {
    "id": 5,
    "name": "Headphones",
    "price": 150.00,
    "is_offer": false
  },
  {
    "id": 6,
    "name": "USB Cable",
    "price": 12.99,
    "is_offer": false
  },
  {
    "id": 7,
    "name": "Desk Lamp",
    "price": 45.00,
    "is_offer": true
  },
  {
    "id": 8,
```



```
    "name": "Chair",
    "price": 199.99,
    "is_offer": false
  },
  {
    "id": 9,
    "name": "Desk",
    "price": 350.00,
    "is_offer": true
  },
  {
    "id": 10,
    "name": "Printer",
    "price": 180.00,
    "is_offer": true
  }
]
```json
```

With 12 items and 5 items per page:

- Page 1 → items 1-5
- Page 2 → items 6-10
- Page 3 → items 11-12

---

## Part 4: Run and Test the Application

### Step 4.1: Start the Server

```
uvicorn app.main:app --reload
```

Open:

```
http://127.0.0.1:8000/items/
```

---

## Part 5: Manual Testing Checklist

☒ Test 1: Default Page Load

Open:

```
/items/
```

Expected:

- Shows first 5 items
- Sort is "Default"
- Page 1 highlighted
- "Previous" disabled
- "Next" enabled

---

### ✅ Test 2: Sort by Name

Open:

```
/items/?sort=name&page=1
```

Expected order:

```
Chair
Desk
Desk Lamp
Headphones
Keyboard
```

Verify:

- "Name" button highlighted
- Pagination still works

---

### ✅ Test 3: Sort by Price

Open:

```
/items/?sort=price&page=1
```

Expected lowest prices first:

```
USB Cable
Mouse
Desk Lamp
```

Keyboard  
Webcam

---

#### ✓ Test 4: Pagination

Open:

```
/items/?page=2
```

Expected:

- Shows items 6-10
  - Page 2 highlighted
  - Previous enabled
  - Next enabled
- 

#### ✓ Test 5: Sorting + Pagination Together

Open:

```
/items/?sort=price&page=2
```

Expected:

- Still sorted by price
- Shows second set of sorted items
- Price button highlighted
- Page 2 highlighted

This test is critical because it verifies state preservation.

---

#### ✓ Test 6: Invalid Page

Open:

```
/items/?page=999
```

Expected:

- Redirects to last valid page
- 

#### ✓ Test 7: Invalid Sort

Open:

```
/items/?sort=banana
```

Expected:

- FastAPI returns validation error

---

## Part 6: How Search Fits Into This Design

If Workshop 1 already included search functionality, you must preserve it.

Example existing API:

```
/items/?search=laptop
```

Now your API may become:

```
/items/?search=laptop&sort=price&page=2
```

### Important Design Principle

Every new feature must preserve existing query parameters.

If user:

1. Searches
2. Sorts
3. Changes page

...the application must remember all three states.

---

### Example State Preservation

Action	URL
Search "desk"	<code>/items/?search=desk</code>
Sort by price	<code>/items/?search=desk&amp;sort=price&amp;page=1</code>
Go to page 2	<code>/items/?search=desk&amp;sort=price&amp;page=2</code>

---

### Common Student Mistake

Wrong:

```

```

Problem:

- Loses current search
- Loses current sort

Correct:

```

```

---

## Part 7: Architecture Lessons

### Lesson 1: Sorting and Pagination Are Server Responsibilities

The browser does not sort the data.

The server:

1. Loads all items
2. Sorts them
3. Slices them
4. Sends only the needed page

---

### Lesson 2: Query Parameters Represent UI State

These:

```
sort=name
page=2
search=desk
```

...are the current UI state.

The URL completely describes what the user is viewing.

---

### Lesson 3: Links Are API Calls

Students often think:

```

```

is "just navigation."

It is actually:

```
HTTP GET request
```

Every button click calls your backend API.

---

## Lesson 4: Preserve State Across Interactions

When adding new features:

- Never destroy existing state
  - Preserve query parameters
  - Keep UI consistent
- 

## Part 8: Final URL Examples

Feature Combination	URL
Default list	<code>/items/</code>
Sort by name	<code>/items/?sort=name</code>
Page 2	<code>/items/?page=2</code>
Sort + page	<code>/items/?sort=price&amp;page=3</code>
Search + sort	<code>/items/?search=desk&amp;sort=name</code>
Search + sort + page	<code>/items/?search=desk&amp;sort=name&amp;page=2</code>

---

## Part 9: Debugging Checklist

If pagination does not work:

- Check `total_pages`
- Check slicing logic
- Check query parameter names
- Check page links

If sorting does not work:

- Check `sort` parameter
- Check `sorted()` function
- Check active button logic

If state disappears:

- Check every link preserves query parameters
- Check template variables passed correctly

If page buttons look wrong:

- Check:

```
range(1, total_pages + 1)
```

---

## Part 10: What Students Learned

After this workshop, students should understand:

- ✓ API-first design
- ✓ Query parameters
- ✓ URL state management
- ✓ Sorting logic
- ✓ Pagination math
- ✓ State preservation
- ✓ Backend/frontend interaction
- ✓ Template-driven UI generation
- ✓ Why links are API calls
- ✓ How multiple features combine cleanly